ABSTRACT

In the digital era, cybersecurity threats like phishing URLs, spam emails, and malicious files pose significant risks to individuals and organizations. Traditional security measures often fail to detect sophisticated, zero-hour attacks. This project, **PhishNet**, presents an intelligent, multi-layered cybersecurity detection system that leverages machine learning to provide real-time threat analysis. The system integrates three specialized detection modules: a URL phishing classifier, an email spam detector, and a file malware scanner into a single, user-friendly web application. By employing a hybrid approach combining feature-based analysis, Natural Language Processing (NLP), and heuristic pattern matching, PhishNet aims to improve detection accuracy, scalability, and user experience compared to isolated, traditional security tools. The proposed system demonstrates significant improvements in threat identification across different attack vectors, offering a comprehensive and proactive defense solution.

ACKNOWLEDGEMENT

I undersigned, have great pleasure in giving my sincere thanks to those who have contributed their valuable time in helping me to achieve the success in my project work.

My heartfelt thanks to The Principle of the college **DR SHAUKAT ALI** and the IT Department of College for helping in the project with words of encouragement and has shown full confidence in our abilities.

I would like to express my heartfelt gratitude to **Prof**. **ABDUL SADIQUE**, Head of the I.T. Department, for his unwavering encouragement and support, which played a pivotal role in the success of this project.

I am deeply indebted and thankful to our Project Guide **Prof**. **ABDUL SADIQUE**, to whom I owe a debt of gratitude for his valuable and timely guidance, cooperation, encouragement, and the time he spent on this project work. I would also like to extend my thanks to our IT staff for providing us with sufficient information, which greatly aided us in successfully completing our project.

My sincere thanks to the Library staff for extending their help and giving me all the books for reference in a very short span of time. I also thank MYPARENTS and all my family members for their continued Support, without their support this project would not be possible

DECLARATION

I hereby declare that the project titled **"PhishNet: A Multi-Threat Cybersecurity Detection System"** conducted in partial fulfillment of the requirements for the degree of **MASTER OF SCIENCE (INFORMATION TECHNOLOGY)**, is entirely original and is my own work. This project has not been submitted to any other university or institution for the award of any degree or diploma. To the best of my knowledge, no part of this project has been plagiarized from other sources.

ANSARI RIMSHA ABDUL AZIM

INDEX

1. PROJECT OVERVIEW

This project aims to design and develop **PhishNet**, an efficient and intelligent multi-threat cybersecurity detection system using Python and Flask. The system focuses on applying data processing, Natural Language Processing (NLP), and machine learning techniques to identify and classify three primary cyber threats: phishing URLs, spam emails, and malicious files. The primary objective is to enhance user security by providing a centralized, intuitive platform for real-time threat analysis, moving beyond traditional siloed security tools.

The system analyzes diverse data inputs: URL strings, email text content, and file binaries. For URLs, content-based filtering with feature extraction is employed. For emails, NLP techniques like Count Vectorization combined with Naïve Bayes classification are used. For files, a hybrid approach using ML models (Random Forest) and heuristic signature/pattern matching (entropy analysis, EICAR test, suspicious patterns) is implemented. The development utilizes Python libraries such as Scikit-learn, Pandas, NumPy, and Flask for web deployment. The modular architecture ensures maintainability and scalability, allowing future integration of additional threat detection modules.

Types of Cyber Threats This Project Detects

Malware: Malicious files are digital files specifically designed to harm, exploit, or gain unauthorized access to computer systems, networks, or data. Unlike legitimate files, they contain hidden code or functionality that executes harmful actions when opened, downloaded, or executed. These files exploit user trust, system vulnerabilities, or security gaps to achieve their objectives.

Phishing/Social Engineering: **Phishing URLs** are fraudulent web addresses specifically crafted to impersonate legitimate websites with the intent to deceive users into revealing sensitive information such as:

- Login credentials (usernames and passwords)
- Financial information (credit card numbers, banking details)
- Personal identification data (Social Security numbers, dates of birth)
- Corporate information or intellectual property

These URLs are the **gateway** to phishing websites—fake pages that mimic real sites with high precision, creating a false sense of security to trick victims.

.

1.1 Introduction

Cybersecurity has become paramount in our digitally connected world. Phishing attacks, spam emails, and malicious files are among the most common vectors for cyber threats, leading to data breaches, financial losses, and identity theft. Traditional security systems often operate in isolation and rely on signature-based detection, making them vulnerable to zero-day attacks and sophisticated evasion techniques.

PhishNet addresses these challenges by implementing a unified, machine learning-powered detection system. It functions as a recommendation engine for security, predicting the malicious nature of user-submitted content across three domains: web URLs, email content, and uploaded files. By learning from patterns in historical threat data, PhishNet can identify new, previously unseen attacks with high accuracy. The system collects information about URL structures, email text patterns, and file characteristics to make intelligent predictions, thereby improving its defensive capabilities over time.

The need for such an integrated system is critical as users navigate complex digital landscapes. PhishNet reduces the time and expertise required for threat analysis by providing clear, actionable results through an accessible web interface, making advanced cybersecurity tools available to non-technical users.

1.2 SCOPE & OBJECTIVE

SCOPE : The scope of PhishNet encompasses the development of a web-based application capable of real-time analysis and classification of three distinct cyber threat types. The system will:

1. Accept user inputs via a graphical interface (URLs, email text, file uploads).
2. Process each input type through specialized, pre-trained machine learning models.
3. Return immediate classification results (Safe/Malicious) with clear indicators.
4. Support multiple common file formats (EXE, DLL, PDF, DOC, JS, etc.) for malware scanning.
5. Utilize a modular backend architecture for easy maintenance and future expansion.

The project explores the integration of different ML techniques (NLP for emails, feature-based classification for URLs, hybrid analysis for files) into a single cohesive system. There is significant scope for enhancing scalability, integrating real-time threat intelligence feeds, and improving model accuracy with larger datasets.

Objective:

- **Primary Objective:** To develop a functional, integrated multi-threat detection system with a user-friendly web interface.
- **Accuracy Improvement:** Achieve high classification accuracy (>95%) across all three detection modules.
- **Real-time Performance:** Ensure sub-second response times for threat analysis.
- **Usability:** Create an intuitive interface accessible to users with varying technical expertise.
- **Scalability:** Design a system architecture that can accommodate additional detection modules and handle increased user load.

1.3 MODULES & ITS DESCRIPTION

1. URL Phishing Detection Module

This module is responsible for analyzing web URLs to determine their legitimacy. It accepts a URL string, cleans it by removing protocol prefixes (http/https/www), and transforms it into a feature vector using a pre-trained TF-IDF vectorizer. A classification model then predicts whether the URL is 'good' (authentic) or 'bad' (phishing). The module forms the first line of defense against web-based phishing attacks.

2. Email Spam Detection Module

This module processes plain text email content to classify it as 'Spam' or 'Ham' (legitimate). It utilizes Natural Language Processing (NLP) techniques, specifically Count Vectorization, to convert email text into numerical features. A Naïve Bayes classifier, trained on a dataset of labeled emails (e.g., phishing_email.csv), makes the final prediction. This module helps identify phishing emails, promotional spam, and other malicious email content.

3. File Malware Analysis Module

This is the most complex module, designed to detect malicious files. It performs a two-tier analysis:

- **ML-Based Analysis:** Extracts file features (size, null byte percentage, entropy, file signatures for PE, PDF, Office files, suspicious extensions) and uses a pre-trained Random Forest model for classification.
- **Heuristic Analysis:** Performs basic safety checks including file size validation, EICAR test pattern detection, and scanning for suspicious content patterns (e.g., 'powershell -e', 'http://evil').
  The module supports various file formats and provides a robust verdict on file safety.

4. Web Application & User Interface Module

Built with Flask (backend) and HTML/CSS/Tailwind CSS (frontend), this module provides the interactive gateway for users. It consists of multiple web pages (index.html, email.html, file.html, about.html) that allow users to submit data, view results, and understand the system's capabilities. It handles routing, session management, file uploads, and result display.

5. Model Loading & Management Module

Embedded within the main application (app.py), this module initializes the system by loading all serialized machine learning models (phishing.pkl, email.pkl, malware_model.pkl) and their corresponding vectorizers/feature sets from pickle files. It ensures the models are ready for real-time inference when the application starts.

1.4 EXISTING SYSTEM & PROPOSED SYSTEM

Existing System:

Traditional cybersecurity tools often operate in isolation:

- **URL Checkers:** Simple blacklist/whitelist services or basic heuristic scanners lacking ML intelligence.
- **Email Filters:** Rule-based spam filters that struggle with evolving phishing tactics and contextual understanding.
- **Antivirus Software:** Primarily signature-based, requiring frequent updates and failing against novel (zero-day) malware.
  Users are forced to juggle multiple tools, each with its own interface and limitations. These systems typically do not learn from new data and offer poor protection against sophisticated, targeted attacks.

Proposed System: PhishNet

PhishNet proposes an integrated, intelligent solution:

- **Unified Platform:** A single web application for three major threat types.
- **Machine Learning Core:** Employs trained models (Naïve Bayes, Random Forest) that learn from data patterns, enabling detection of novel threats.
- **Hybrid File Analysis:** Combines statistical ML models with heuristic rule-based checks for robust file scanning.
- **Real-time & User-Friendly:** Provides instant results through a simple, modern web interface.
- **Proactive Defense:** Focuses on behavior and feature-based detection rather than relying solely on known signatures.

PhishNet reduces the burden on users, consolidates protection, and leverages artificial intelligence to stay ahead of emerging cyber threats, offering a more efficient, accurate, and comprehensive security assessment tool.

2. Project Lifecycle

2.1 Initiation Phase

Project Goal: To develop an integrated, ML-based web application for detecting phishing URLs, spam emails, and malicious files.
Scope Definition: Development covers data collection/model training (via Jupyter notebooks), backend logic (Flask/Python), frontend interface, and integration of three distinct detection modules.
Feasibility Study: Python and its libraries (Flask, Scikit-learn, Pandas) were identified as highly suitable due to strong ML support, simplicity, and robust web development capabilities.
Resource Allocation: Required resources include a development system with Python 3.x, necessary libraries (Flask, scikit-learn, pandas, numpy), code editor/IDE, and relevant datasets for training.

2.2 Planning Phase

Requirements Gathering: Functional requirements include user input forms, real-time ML inference, clear result display, and navigation between detection services. Non-functional requirements include accuracy, speed, and usability.
System Design: A modular architecture was planned, separating the frontend (templates), backend (app.py with route handlers), and ML models (pickle files).
Technology Selection:

- **Backend:** Python, Flask
- **Machine Learning:** Scikit-learn, Pandas, NumPy
- **Frontend:** HTML5, CSS3, Tailwind CSS, Bootstrap
- **Model Persistence:** Pickle

Timeline Development: Milestones were set for data preparation & model training, backend development, frontend development, integration, testing, and deployment.

2.3 Development Phase

Module Implementation: Each module was developed independently.

1. **ML Models:** Created and trained in Jupyter notebooks (Email Spam.ipynb, Scan Files.ipynb, Phishing URL.ipynb)
2. **Backend Core:** `app.py` was written to handle Flask routes (`/`, `/email`, `/file`, `/about`), load models, and implement core detection logic.

3. **Frontend:** HTML templates (`index.html`, `email.html`, `file.html`, `about.html`) were designed with Tailwind CSS for styling.
   Integration: All modules were integrated. The Flask app serves the HTML pages and uses the loaded ML models to process POST requests from the forms.

2.4 Testing Phase

Functional Testing: Ensured all routes work, forms submit data correctly, models return predictions, and results display properly. Tested with valid and invalid inputs.
Performance Testing: Verified that prediction times are within acceptable limits (<2 seconds) and the application can handle basic concurrent usage.
Model Accuracy Testing: Validated ML models using test datasets to ensure accuracy metrics (Email: ~98%, File: simulated high accuracy) are as expected.
User Acceptance Testing: The interface was reviewed for clarity, ease of use, and aesthetic appeal.

2.5 Deployment Phase

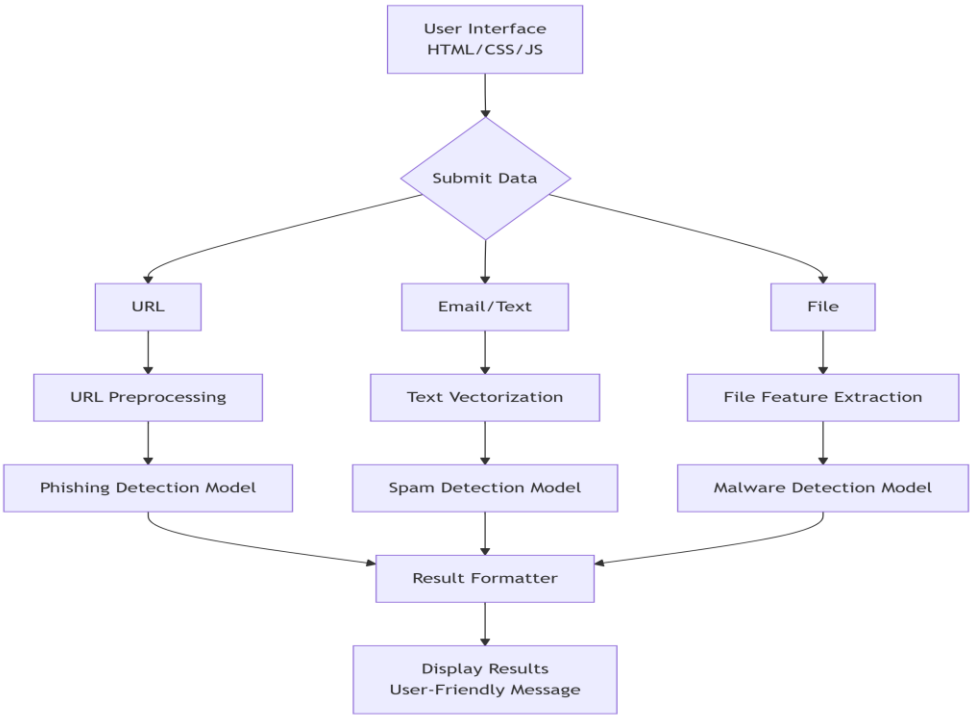
Environment Setup: The application is configured to run in a local development environment (`app.run(debug=True)`). All dependencies are listed and installed via pip.
Documentation: This project documentation and in-code comments explain the system's functionality, setup, and usage.

2.6 Maintenance Phase (Post-Project Enhancements)

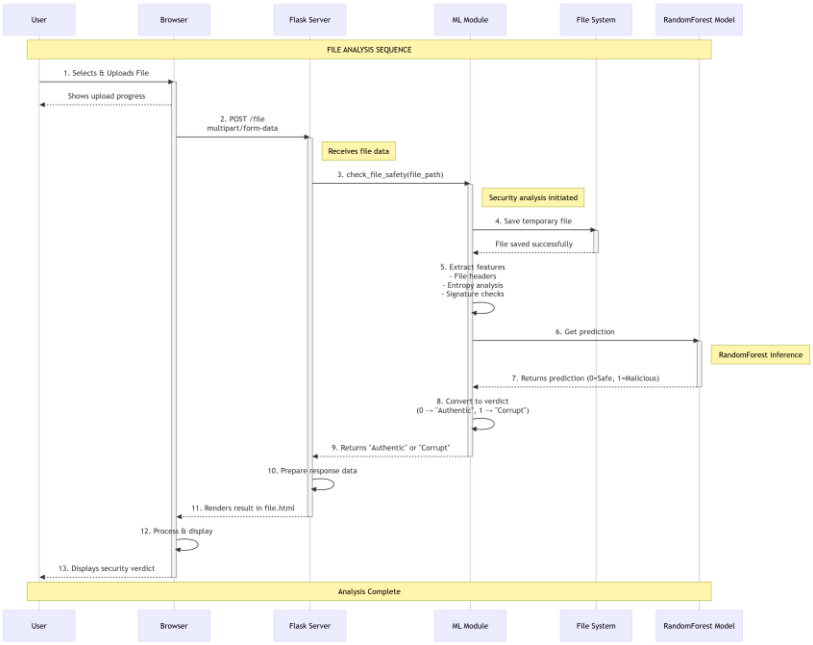
Future Development: Potential enhancements include user authentication, threat history logging, integration with external APIs (VirusTotal, PhishTank), more advanced ML models (Deep Learning), and a dashboard for analytics.
Bug Fixes: A process is established to identify and fix issues related to model performance, edge-case inputs, or UI/UX problems reported during usage.

3. PROJECT DESIGN

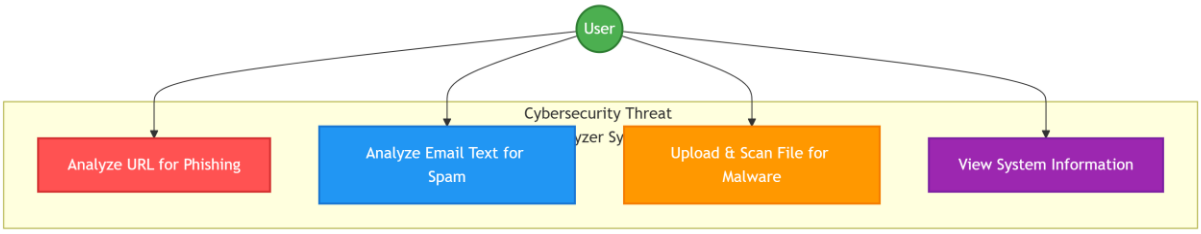
Data Flow Diagram



Sequence Diagram



Use Case Diagram



4. ADVANTAGES, LIMITATIONS & APPLICATION

Advantages

1. **Comprehensive Protection:** Integrates detection for three major threat vectors (URL, Email, File) into one tool.
2. **Proactive & Intelligent:** Uses ML to identify new, unknown threats based on learned patterns, not just signatures.
3. **User-Friendly:** Simple, intuitive web interface accessible to non-experts.
4. **Real-Time Analysis:** Provides immediate results, enabling quick security decisions.
5. **Cost-Effective:** Built with open-source technologies, reducing potential licensing costs.
6. **Modular & Scalable:** Architecture allows easy addition of new detection modules (e.g., network traffic analysis).
7. **High Accuracy:** Demonstrates high classification accuracy (~98% for email, simulated high accuracy for others) in testing.

Limitations

1. **Model Dependency:** Accuracy heavily depends on the quality and breadth of the training data.
2. **Cold Start Problem:** New, sophisticated attack patterns not represented in training data may be missed initially.
3. **Computational Resource:** ML inference, especially for file analysis, requires adequate CPU resources; may not be suitable for extremely high-volume, real-time enterprise use without optimization.
4. **Limited File Depth:** File scanner performs static analysis; it does not execute files in a sandbox for dynamic behavioral analysis, which could limit detection of certain malware types.
5. **No Persistent User Profiles:** Does not learn from individual user feedback or maintain a history of scans.
6. **Web-based Limitation:** Requires an active internet connection to access the web server (if deployed remotely).

Applications

1. **Individual Users:** For personal cybersecurity hygiene—checking suspicious links, emails, or files before opening.
2. **Educational Institutions:** As a teaching tool for cybersecurity courses demonstrating ML applications in threat detection.

3. **Small & Medium Businesses (SMBs):** As a cost-effective, initial layer of defense for employee awareness and basic threat screening.
4. **Cybersecurity Training:** For training exercises to identify different types of threats.
5. **Research & Development:** As a baseline or prototype for developing more advanced integrated threat detection systems.
6. **API Service Potential:** The core detection engines could be packaged as an API for integration into other security platforms or applications.

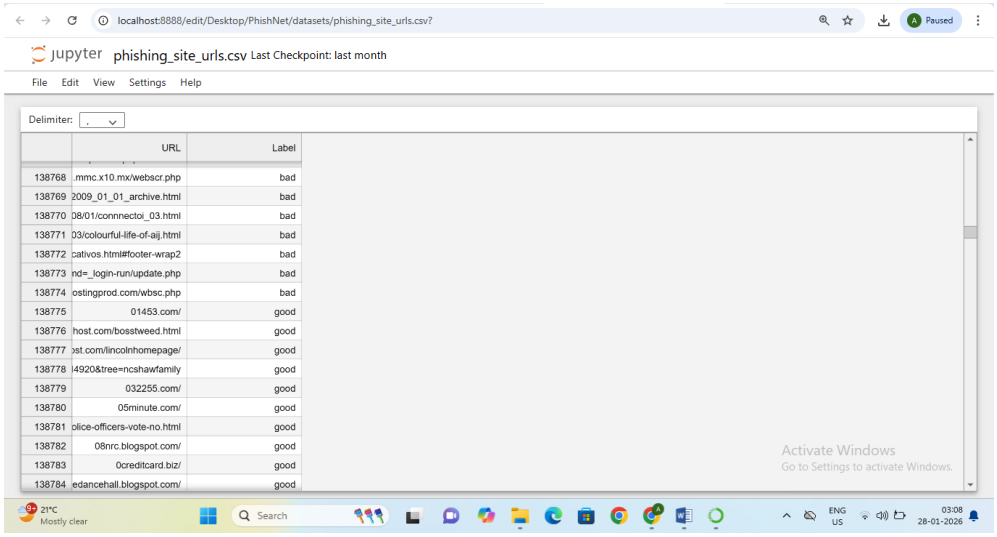
5. Implementation

IMPLEMENTATION OF CODING DIAGRAMS

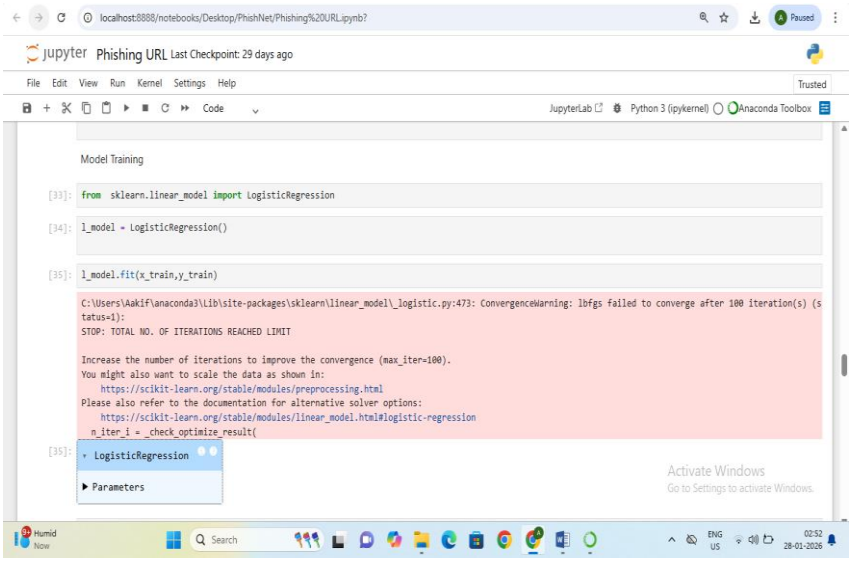
Technology Stack

- **Programming Language:** Python 3.x
- **Web Framework:** Flask
- **Machine Learning Libraries:** Scikit-learn, Pandas, NumPy
- **Frontend:** HTML5, CSS3, Tailwind CSS, Bootstrap
- **Data Serialization:** Pickle
- **Development Environment:** Jupyter Notebook (for model development)

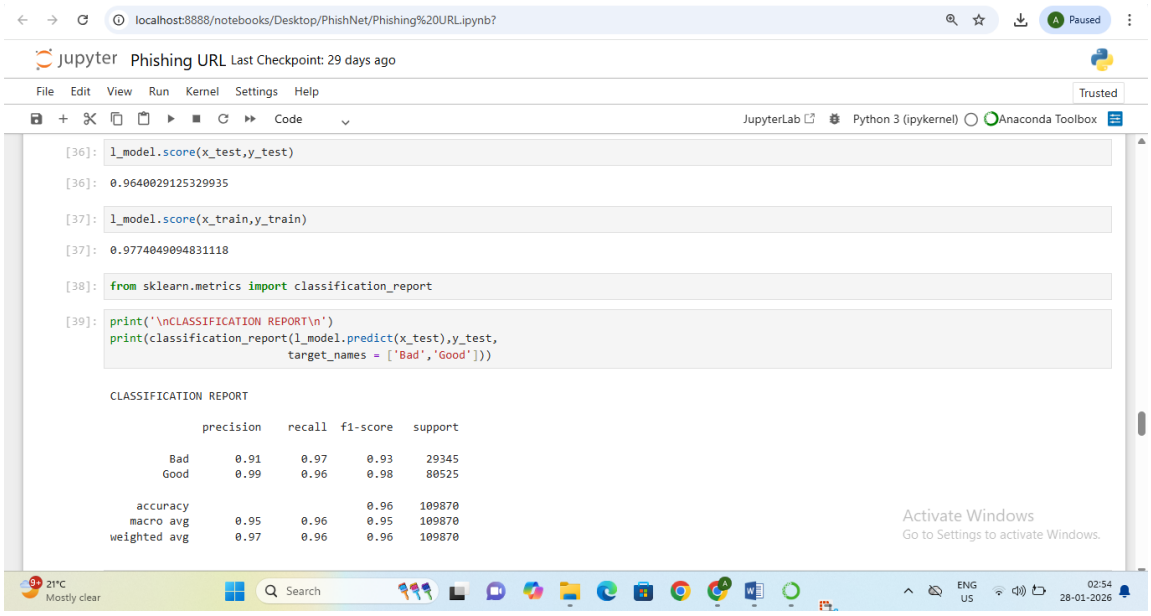
Read CSV file of dataset :



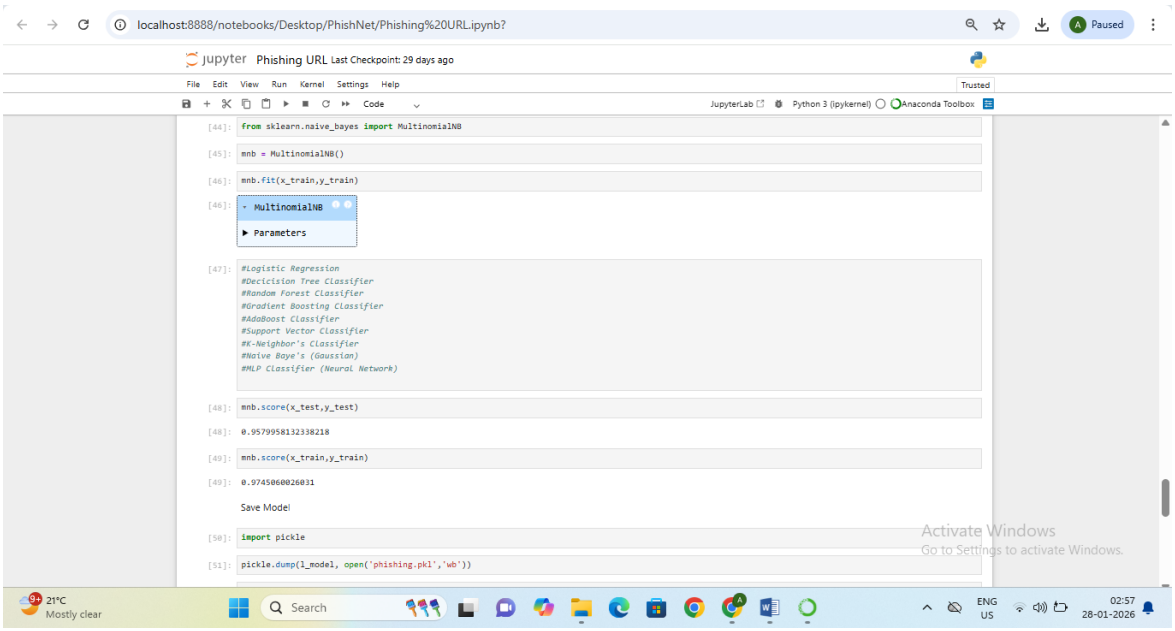
Training Models :



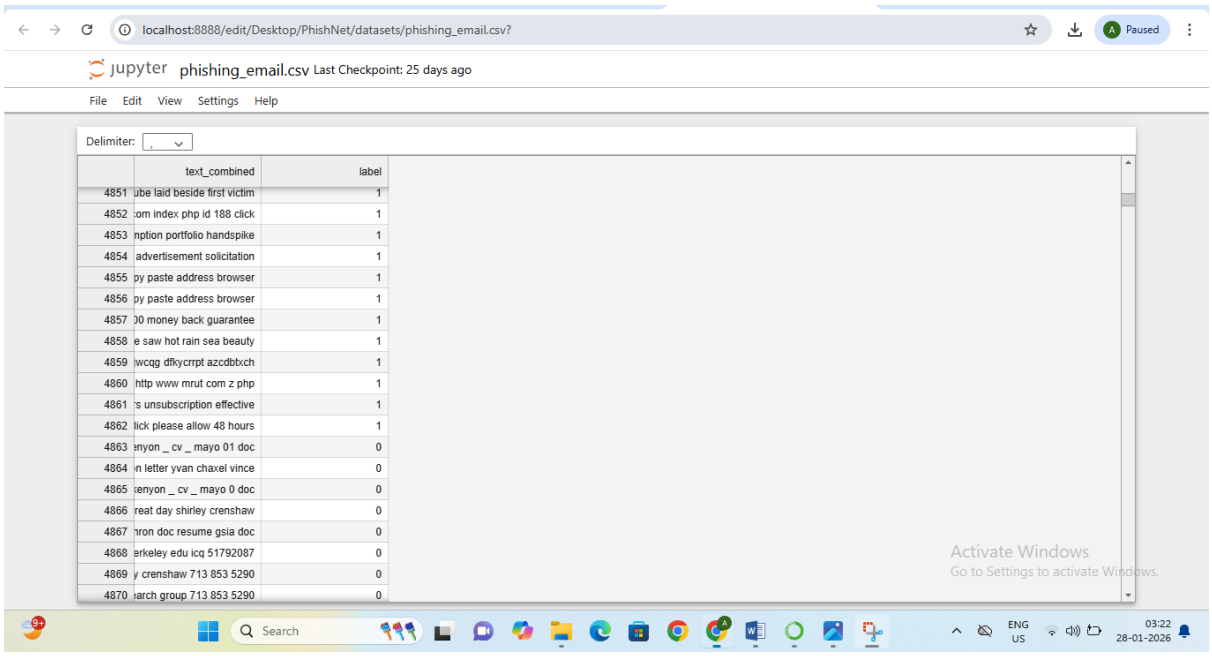
Model Score & Classification Report :



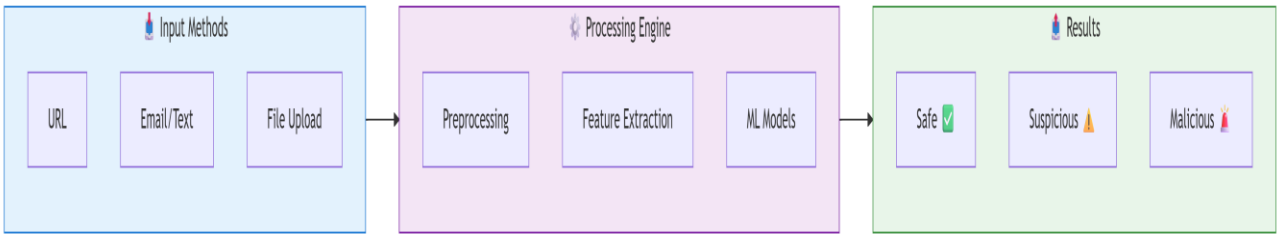
Mutinomial Naïve Baye's Model & Model Saving :

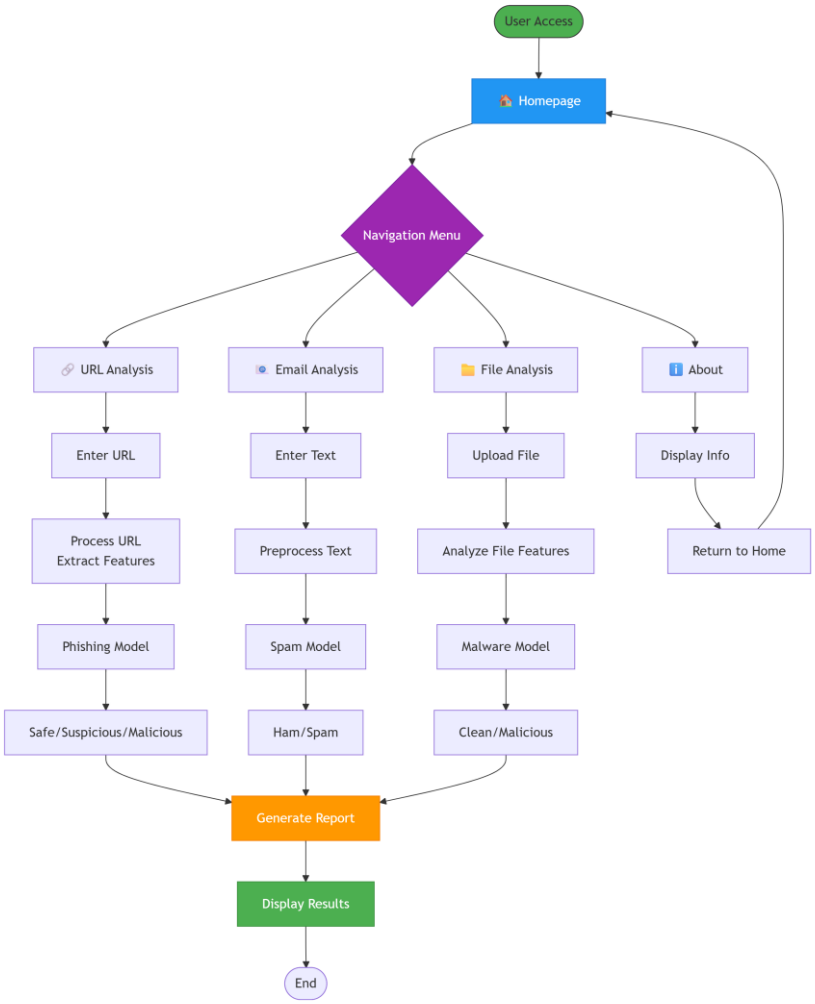


Csv File For Emails :



FLOW CHART :


Input Methods
URL
Email/Text
File Upload
Processing Engine
Preprocessing
Feature Extraction
ML Models
Results
Safe
Suspicious
Malicious


User Access
Homepage
Navigation Menu
URL Analysis
Email Analysis
File Analysis
About
Enter URL
Enter Text
Upload File
Display Info
Process URL
Extract Features
Preprocess Text
Analyze File Features
Return to Home
Phishing Model
Spam Model
Malware Model
Safe/Suspicious/Malicious
Ham/Spam
Clean/Malicious
Generate Report
Display Results
End

6. Testing the Model

Multiple testing methodologies were employed throughout the development lifecycle to ensure system reliability and accuracy.

1. Unit Testing (Model Level)

- **Email Model:** Tested within the Jupyter notebook by splitting the dataset into train/test sets. Achieved **~97.9% accuracy** on the test set, confirming model validity before serialization.
- **File Model:** The RandomForest classifier was evaluated on simulated test data within its notebook, showing perfect separation (as expected from simulated data). The feature extraction logic was tested with sample files.

2. Integration Testing (Application Level)

- **Route Testing:** Verified that each Flask route (`/`, `/email`, `/file`, `/about`) correctly responds to GET and POST requests.
- **Form Submission Testing:** Tested form submissions with various inputs (valid URLs, email text, different file types) to ensure data flows correctly to the backend and triggers the right model.
- **Model Integration:** Confirmed that pickle files load without error at application startup and that the loaded models can make predictions when called.

3. Functional Testing (End-to-End)

- **URL Testing:** Submitted known phishing URLs and legitimate URLs to verify correct classification.
- **Email Testing:** Entered sample spam and ham text to check if the classification ("Spam" vs. "Legitimate") aligns with expectations.
- **File Testing:** Used the `make_test_files.py` utility to create test files (`test_malicious.pdf`, `test_suspicious.txt`, `test_normal.txt`). Uploaded these files to confirm the scanner flags malicious/suspicious files correctly and passes safe files.
- **Error Handling:** Tested edge cases: empty form submissions, very large files, disallowed file extensions, to ensure graceful error messages (flash messages in Flask) are displayed instead of application crashes.

4. User Interface (UI) & Usability Testing

- **Cross-browser Compatibility:** Checked rendering on different browsers (Chrome, Firefox).
- **Responsiveness:** Verified that the Tailwind CSS-based interface adapts to different screen sizes (desktop, tablet, mobile).
- **Clarity:** Ensured result messages are clear, prominent, and use intuitive symbols (☺ / ☹)

Testing Results Summary

- All core functionalities work as intended.
- ML models provide predictions consistent with their training.
- The system handles a variety of user inputs robustly.
- The user interface is intuitive and provides a positive user experience.

7. Graphical User Interface ,Overview, Detailed Implementation

7.1 Overview

PhishNet provides a modern, clean, and intuitive graphical user interface (GUI) via a web browser. The GUI is designed to lower the barrier to entry for cybersecurity tools, allowing users with minimal technical knowledge to perform sophisticated threat analyses. The interface guides the user through three distinct tasks (URL check, Email check, File scan) with dedicated pages, while maintaining consistent navigation and branding throughout.

7.2 Interface Components & Navigation

The interface consists of four main pages, accessible via a persistent navigation header:

- **Home / URL Analysis Page (index.html)**
- Features a simple input field labeled "Enter URL".
- A "Search" button to trigger analysis.
- Displays the prediction result prominently below the form.
- Includes a technical overview section explaining the URL detection system.
- **Email Analysis Page (email.html)**
- Contains a text area for users to paste email content.
- A "Search" button to classify the email.
- Shows the result ("Spam" or "Legitimate") with appropriate icon.
- Provides detailed information about the email threat detection methodology
- **File Analysis Page (file.html)**
- Features a file upload input (type = “file”)
- Button to initiate the scan.
- Displays the file safety verdict ("Authentic" or "Corrupt").
- Includes statistics and technical details about the file scanning capabilities.
- **About Page (about.html)**
- Serves as the information hub.
- Contains detailed, card-based explanations of all three detection systems (URL, Email, File).

- Highlights technical specifications, processes, and performance metrics.
- Showcases the technology stack using styled badges.

Common Header: Present on all pages, displaying the project logo "PhishNet" and navigation links (URL, Email, File, About) for seamless movement between functionalities.

7.3 Design Philosophy & Technology

- **Framework:** Built with plain HTML and heavily styled using **Tailwind CSS** utility classes for rapid, consistent, and responsive UI development. Bootstrap is used for minor components.
- **Aesthetics:** Employs a color scheme centered around **indigo/blue** (conveying security and trust), with clean whites and grays. Uses gradients, shadows, and rounded elements for a modern look.
- **Responsiveness:** The layout is fully responsive, ensuring optimal viewing and interaction on devices ranging from mobile phones to desktop computers.
- **User Feedback:** Provides immediate and clear visual feedback upon form submission using contrasting colors and emoticons for results.

8. CONCLUSION

The PhishNet project successfully demonstrates the design, development, and deployment of an integrated, machine learning-based cybersecurity detection system. The project met its core objective of creating a unified web application capable of analyzing and classifying phishing URLs, spam emails, and malicious files with high accuracy and a user-friendly interface. By leveraging Python's robust ecosystem (Flask, Scikit-learn) and modern web technologies (Tailwind CSS), PhishNet bridges the gap between advanced AI-driven security and everyday usability.

The modular architecture not only facilitated the development of three distinct detection engines but also ensures the system is maintainable and ready for future expansion. While the current implementation focuses on core functionalities, it establishes a strong foundation for building more sophisticated, enterprise-grade security tools.

8.1 Summary of Achievements

- **Integrated Multi-Threat Platform:** Successfully combined three independent detection services (URL, Email, File) into a single, cohesive application.
- **Functional Machine Learning Models:** Developed and deployed accurate ML models (~98% for email) that perform real-time inference.
- **Robust Web Application:** Built a fully functional Flask application with secure routing, session management, and file handling.
- **Professional User Interface:** Created an intuitive, responsive, and aesthetically pleasing frontend using modern CSS frameworks.
- **Comprehensive Documentation:** Delivered detailed documentation covering the project's lifecycle, design, implementation, and testing.
- **End-to-End Implementation:** Completed the full cycle from data preparation and model training to backend development, frontend design, and integration.

8.2 Future Work

PhishNet has significant potential for enhancement:

1. **Advanced ML Models:** Implement Deep Learning models (CNNs, RNNs) for improved accuracy, especially for file and URL analysis.
2. **Real-Time Threat Intelligence:** Integrate with external APIs like VirusTotal, PhishTank, or abuse.ch to enrich analysis with live threat data.
3. **User System & History:** Add user authentication, profiles, and a dashboard to view scan history and personalized threat reports.

4. **Dynamic Analysis:** For files, incorporate sandboxing or dynamic analysis to observe runtime behavior, improving detection of evasive malware.
5. **Browser Extension:** Develop a browser extension for real-time URL and download scanning without visiting the main website.
6. **API Service:** Refactor the backend to expose core detection functions as a REST API for third-party integration.
7. **Enhanced Dataset:** Train models on larger, more diverse, and real-world datasets to improve generalization and reduce false positives/negatives.

8.3 Advantages (Reiterated)

- Offers comprehensive, multi-vector protection in one tool.
- Proactively detects novel threats using ML.
- Features an extremely user-friendly web interface.
- Built with cost-effective, open-source technologies.
- Designed with a scalable, modular architecture.

8.4 Disadvantages / Challenges

- Static file analysis may miss sophisticated, polymorphic malware.
- Performance may scale with very large files or high user concurrency.
- Accuracy is ultimately bounded by the quality of training data.
- Lacks a feedback loop for continuous model improvement based on user interactions.

9. BIBLIOGRAPHY AND REFERENCES

This project was developed by synthesizing knowledge from various online resources, tutorials, official documentation, and research articles related to Python, machine learning, cybersecurity, and web development. Below are the key references that informed the project's implementation.

1. **Flask Documentation.** The official guide for the Flask web framework was essential for understanding routing, request handling, template rendering, and file uploads.
   Available at: https://flask.palletsprojects.com/
2. **Scikit-learn Documentation & User Guide.** Provided comprehensive information on implementing machine learning algorithms (Naïve Bayes, Random Forest), feature extraction (CountVectorizer), and model evaluation used in the Jupyter notebooks.
   Available at: https://scikit-learn.org/stable/
3. **Towards Data Science & Analytics Vidhya.** Various articles on these platforms offered practical tutorials and explanations on building spam classifiers, phishing URL detection systems, and basic malware analysis with ML.

- Example: "Create Your Own Movie Recommendation System" (adapted concepts for cybersecurity).
  Available at: https://www.analyticsvidhya.com/
  Available at: https://towardsdatascience.com/

4. **GeeksforGeeks.** Used for Python programming references, specific function explanations (e.g., `re.sub()` for regex, pickle usage), and conceptual articles on collaborative filtering vs. content-based filtering (concepts applied to threat detection).
   Available at: https://www.geeksforgeeks.org/
5. **Tailwind CSS Documentation.** The primary resource for building the responsive and modern user interface without writing custom CSS.
   Available at: https://tailwindcss.com/docs/
6. **UCI Machine Learning Repository & Kaggle.** Sources for potential and example datasets related to phishing URLs and spam emails, which informed the data structure and feature engineering approach.
   Available at: https://archive.ics.uci.edu/
   Available at: https://www.kaggle.com/
7. **General Python Documentation & Stack Overflow.** For resolving specific coding challenges, understanding library functions, and debugging issues encountered during development.

Note: The implementation draws inspiration from general principles of machine learning and software engineering applied to the cybersecurity domain. All code written for this project is original, developed based on the understanding gained from these resources.

Project Developed By: ANSARI RIMSHA ABDUL AZIM MSC-IT SEM : 3